

12. Faster R-CNN

Overview

Faster R-CNN is a two-stage detector: a Region Proposal Network proposes boxes, then RoI pooling classifies and refines them. This module explains anchor scales, IoU matching, RPN losses, and inference pipelines you will run in the lab.

Lab: Lab 12.

Contents

1	Object Detection Problem	2
2	Region Proposal Network	2
3	RoI Pooling and Head	2
4	Multi-Task Loss	2
5	Anchor Matching	2
6	Feature Pyramid Networks	2
7	Lab	3
8	RoI Align	3
8.1	Anchor Design	3
9	Extended Intuition	3
10	Numerical Walkthrough	4
11	PyTorch Implementation Notes	4
12	Local GPU Constraints	4
13	Debugging Checklist	4
13.1	Feature Pyramid and Inference Pipeline	5
14	Practice Problems	5

Module track

This module starts a new track. Later modules refer back using **Module NN** labels.

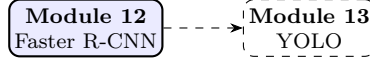


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Object Detection Problem

Given image $\mathbf{I}$, predict bounding boxes $\mathbf{b}_k = (x, y, w, h)$ and class labels c_k . Two-stage detectors trade speed for accuracy; Faster R-CNN [5] is the canonical two-stage pipeline.

2 Region Proposal Network

RPN slides a small network over conv feature map $\mathbf{F}$:

$$p_i = \sigma(\mathbf{W}_p \cdot \text{anchor}_i), \quad \mathbf{t}_i = \mathbf{W}_t \cdot \text{anchor}_i, \quad (1)$$

where anchors are preset boxes at multiple scales/aspect ratios.

Note. RPN shares backbone features with the detector, unlike earlier R-CNN variants with external proposals.

3 RoI Pooling and Head

Region of Interest (RoI) Pooling extracts fixed-size features per proposal:

$$\mathbf{f}_k = \text{RoIPool}(\mathbf{F}, \mathbf{b}_k) \in \mathbb{R}^{7 \times 7 \times d}. \quad (2)$$

Bounding-box regressor refines $\mathbf{b}_k$; classifier outputs $P(c \mid \mathbf{b}_k)$.

4 Multi-Task Loss

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda \mathcal{L}_{\text{box}}, \quad (3)$$

with $\mathcal{L}_{\text{box}}$ smooth L1 on (t_x, t_y, t_w, t_h) offsets and $\mathcal{L}_{\text{cls}}$ cross-entropy.

5 Anchor Matching

IoU with ground truth determines labels:

$$\text{IoU}(\mathbf{b}, \mathbf{b}^{\text{gt}}) = \frac{|\mathbf{b} \cap \mathbf{b}^{\text{gt}}|}{|\mathbf{b} \cup \mathbf{b}^{\text{gt}}|}. \quad (4)$$

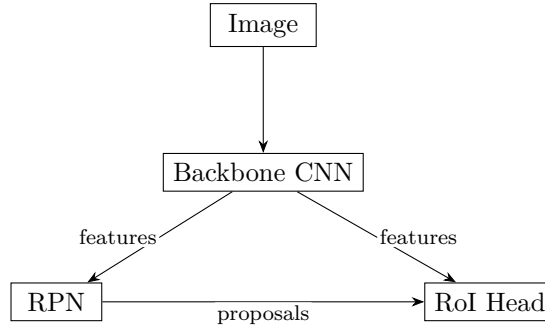
IoU ≥ 0.7 positive; IoU < 0.3 negative.

6 Feature Pyramid Networks

FPN merges multi-scale feature maps for small-object detection, used in Mask R-CNN (**Module 14**).

Table 1: Detector families.

Method	Stages	Typical FPS
Faster R-CNN	2	5 to 15
YOLO (Module 13)	1	30 to 150

**Figure 2:** Faster R-CNN pipeline.

7 Lab

Lab 12 fine-tunes Faster R-CNN on a custom dataset; compare mAP to YOLO in **Module 13** (mAP definition).

8 RoI Align

Bilinear sampling at continuous locations fixes quantization error of RoI Pool:

$$\mathbf{f}(i, j) = \sum_{(p, q)} \text{bilinear}(\mathbf{F}, h_{ij}^{(p, q)}). \quad (5)$$

Mask R-CNN (**Module 14**) requires Align for pixel-accurate masks.

8.1 Anchor Design

Aspect ratios $\{0.5, 1, 2\}$ and scales $\{128^2, 256^2, 512^2\}$ cover COCO object shapes.

9 Extended Intuition

Faster R-CNN is a two-stage detector: first propose class-agnostic regions, then classify and refine each region. The Region Proposal Network (RPN) slides a small network over a convolutional feature map and predicts objectness plus box deltas at each anchor. Anchors are preset boxes at multiple scales and aspect ratios; the network learns which anchors align with objects.

RoI Pooling (or RoI Align in Mask R-CNN, **Module 14**) crops fixed-size features per proposal for the detection head. Multi-task loss combines RPN objectness/bbox regression with final class classification and bbox refinement. Two-stage designs tend to be more accurate than one-stage YOLO (**Module 13**) but slower at inference.

Training requires careful label assignment: anchors with $\text{IoU} > 0.7$ to a ground-truth box are positive, < 0.3 negative, between ignored. Batch sampling balances foreground and background anchors so the RPN does not predict "background" everywhere.

10 Numerical Walkthrough

COCO-style mini setup: image 800×800 , ResNet-50 FPN backbone, feature stride 16, so map size 50×50 .

Three scales $\times$ three aspect ratios = 9 anchors per spatial location $\Rightarrow 50 \cdot 50 \cdot 9 = 22500$ anchors per image. RPN cls head outputs 2 channels per anchor (bg/fg); bbox head outputs 4 deltas per anchor.

Suppose one anchor matched to GT box with IoU 0.82: cls target foreground, reg target $\Delta = (t_x, t_y, t_w, t_h)$ encoded relative to anchor. Mini-batch 2 images on 1650: each image resized so long side 800, short side ≥ 600 .

`torchvision.models.detection.fasterrcnn_resnet50_fpn` with 91 COCO classes: forward returns `loss_dict` during training. Typical total loss 0.5 to 2.0 after a few epochs on small custom set (10 to 50 images per class) with `lr = 0.005`, SGD, momentum 0.9.

11 PyTorch Implementation Notes

- `fasterrcnn_resnet50_fpn(weights=..., num_classes=N+1)` where N is foreground classes plus background handled internally.
- Custom dataset returns `(image, target)` with `target={"boxes": FloatTensor, "labels": Int64Tensor}` in `xyxy` pixel coords.
- Transforms must update boxes when flipping: swap x coordinates relative to image width.
- `model.train()` returns losses; `model.eval()` returns list of dicts with `boxes`, `labels`, `scores`.
- Lower default `lr` for fine-tune: 0.005 SGD or $1e-4$ Adam on small datasets to avoid destroying pretrained backbone.

```
images, targets = next(iter(train_loader))
loss_dict = model(images, targets)
loss = sum(loss for loss in loss_dict.values())
loss.backward()
```

12 Local GPU Constraints

Faster R-CNN ResNet-50 FPN at 800 px, $B = 1$ to 2 is realistic on 4 GB; $B = 4$ often OOMs. Use smaller backbone (`mobilenet_v3_large_fpn`) for lab-scale data if needed.

Mixed precision (`autocast`) helps; torchvision detection models support it in recent PyTorch versions. Freeze backbone initially for 1 epoch on tiny datasets, then unfreeze with lower `lr`.

13 Debugging Checklist

- Zero detections at inference: score threshold too high (default 0.05), or model never trained (wrong eval mode).
- Loss NaN: bad box coordinates (negative width/height) or labels starting at 0 instead of 1 for COCO heads.
- All background predictions: anchor matching broken after custom resize without updating boxes.
- OOM: reduce image scale to 512 long side or use $B = 1$.
- mAP stuck at 0: IoU threshold mismatch between train labels and eval metric code.

13.1 Feature Pyramid and Inference Pipeline

ResNet-FPN backbones output multi-scale feature maps $\{P_2, P_3, P_4, P_5, P_6\}$ with strides $\{4, 8, 16, 32, 64\}$ pixels relative to the input image. Small objects live on high-resolution maps (P_2, P_3); large objects anchor on coarser levels. The RPN runs the same 3×3 conv head on every level, which is why anchor scales must differ per pyramid level rather than using one global anchor size. After RPN proposes thousands of boxes, the RoI head subsamples a fixed mini-batch (512 RoIs typical) with a 1:3 foreground-to-background ratio so classification loss does not drown in easy negatives.

At inference the pipeline is: resize image, backbone forward, RPN proposals, RoI align, class scores + box refinements, score threshold, class-wise NMS. NMS keeps the highest-scoring box and suppresses any remaining box with IoU above a threshold (often 0.5) to the kept box; repeat per class. Score threshold default 0.05 in torchvision is low on purpose; raise to 0.3 to 0.5 for cleaner visualizations on custom datasets with few classes. Box regression predicts deltas (t_x, t_y, t_w, t_h) applied to proposal (x_a, y_a, w_a, h_a) :

$$x = x_a + t_x w_a, \quad y = y_a + t_y h_a, \quad w = w_a \exp(t_w), \quad h = h_a \exp(t_h). \quad (6)$$

Decoding with wrong anchor reference (center vs corner) produces boxes shifted by half the image; always verify against torchvision reference on one GT annotation.

Two-stage latency on a 1650 at 800px is often 80 to 150 ms per image ($B = 1$), dominated by backbone and RPN, not the RoI head. For deployment, export to TorchScript or ONNX only after confirming `model.eval()` NMS behavior matches training eval scripts. Compare against YOLO (**Module 13**) on the same val set using identical score thresholds and IoU definitions; mAP differences of 5+ points often trace to NMS or resize policy, not model capacity. RPN training uses binary cross-entropy for objectness and Smooth L1 for box regression; only positive anchors contribute to reg loss, which is why anchor matching thresholds dominate early training dynamics. Hard negative mining in the RoI stage subsamples background proposals with high loss so the classifier sees informative negatives, not random easy background patches. Multi-scale training (random short side 640 to 800) improves robustness on custom datasets with variable resolution but increases training time 20 to 40%; fix scale at inference for reproducible benchmarks. COCO pretrained heads expect 91 classes; replacing `box_predictor` and `cls_score` for N custom classes requires re-init of final layers while keeping FPN backbone weights. Backbone freeze schedule: epoch 0 to 1 train head only (lr 10^{-3}), epoch 2+ unfreeze stage4 with lr 10^{-4} on backbone; prevents destroying ImageNet features on tiny datasets. Evaluate with `pycocotools` when comparing to published benchmarks; custom IoU loops often disagree on box format or crowd ignore flags. Visualize top-50 RPN proposals by objectness; if all cluster in image center, anchor scales mismatch your object sizes. Test-time horizontal flip TTA: run detector on image and mirror, map boxes back, merge with NMS; gains 1 to 2 mAP on small val sets at $2\times$ latency. Minimum viable custom dataset: 50 to 100 images per class with varied backgrounds; below that, prefer transfer from COCO pretrained head without changing backbone lr. Log `loss_dict` keys each epoch; RPN `loss_objectness` stuck above 0.5 while `loss_box_reg` drops means proposals never align with objects.

14 Practice Problems

Problem 1. Given anchor (x_a, y_a, w_a, h_a) and GT box, write encoding for $t_w = \log(w_{gt}/w_a)$.

Problem 2. Count anchors for 40×40 feature map with 5 scales/ratios per location.

Problem 3. Fine-tune on 3-class custom dataset; report mAP@0.5 vs YOLO in **Module 13**.

Problem 4. Visualize RPN proposals before second stage on one image (top 100 by objectness).

Train in **Lab 12**; compare speed/accuracy tradeoff with **Module 13**; segmentation extends in **Module 14**.

Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

Table 3: Certified topic map for Module 12.

Topic	Primary source	Where in this PDF
Faster R-CNN / RPN	[5]	Section 2
R-CNN lineage	[1]	Section 1
ResNet backbone	[2]	Section 6
Feature pyramid context	[3]	Extension section

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

References

- [1] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [3] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, et al. Feature pyramid networks for object detection. In *CVPR*, 2017.
- [4] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, 2016.
- [5] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015.